Livox MID-360 Step-by-Step Guide :root { --bg: #f4f8fb; --paper: #ffffff; --ink: #1d2733; --muted: #5b6875; --line: #d7e1ea; --accent: #17624b; --accent-soft: #e8f4ef; --warn: #8a5a00; --warn-soft: #fff6df; } body { margin: 0; background: linear-gradient(180deg, #eaf3fb 0%, #f7fbff 18%, #ffffff 100%); color: var(--ink); font-family: "Segoe UI", Roboto, Helvetica, Arial, sans-serif; } main { max-width: 980px; margin: 0 auto; padding: 40px 24px 72px; } h1, h2, h3 { line-height: 1.2; color: #102235; } h1 { font-size: 2.4rem; margin-bottom: 0.35rem; } h2 { margin-top: 2.2rem; border-bottom: 2px solid var(--line); padding-bottom: 0.35rem; } p, li { line-height: 1.58; } .lead { color: var(--muted); max-width: 58rem; font-size: 1.05rem; } .box, .warn { border: 1px solid var(--line); border-radius: 12px; padding: 14px 16px; margin: 1rem 0; } .box { background: var(--accent-soft); border-left: 6px solid var(--accent); } .warn { background: var(--warn-soft); border-left: 6px solid #d9a31a; } .step { background: var(--paper); border: 1px solid var(--line); border-radius: 14px; padding: 18px 18px 14px; margin: 1rem 0; box-shadow: 0 10px 24px rgba(16, 34, 53, 0.05); } .step h3 { margin-top: 0; } table { width: 100%; border-collapse: collapse; margin: 1rem 0; } th, td { border: 1px solid var(--line); padding: 0.6rem 0.7rem; text-align: left; vertical-align: top; } th { background: #eef4f8; } pre { background: #111821; color: #edf2f7; padding: 14px 16px; border-radius: 10px; overflow-x: auto; } code { font-family: "SFMono-Regular", Consolas, "Liberation Mono", Menlo, monospace; } ol, ul { padding-left: 1.25rem; }
Livox MID-360 Step-by-Step Guide

This guide walks through Livox MID-360 bring-up in this repository from a fresh host: network setup, SDK install, configuration, live verification, ROS driver usage, and the local Python viewer and SLAM demos.

This guide is based on the local repo files docs/lidar_cheatsheet.html, docs/lidar_docs.html, scripts/navigation/slam/livox2_python.py, scripts/navigation/slam/livox_python.py, scripts/sensors/live_points.py, scripts/navigation/slam/live_slam.py, and scripts/navigation/slam/mid360_config.json.

Before You Start

ItemExpected Value

LiDAR modelLivox MID-360

LiDAR IP192.168.123.120

Typical host IP192.168.123.222

Recommended SDKLivox-SDK2

Repo root in this workspace/home/melodse/ef_ws/g1

The local Python viewer and SLAM scripts assume the host is on the 192.168.123.0/24 network and that the LiDAR points at that host IP in its JSON config.

Step 1. Connect the LiDAR Network

Goal

Place the host PC and MID-360 on the same subnet.

- Connect Ethernet from the G-1 shoulder port or Livox network path to your PC.

- Find your network interface name with ip addr.

- Assign the host interface an address like 192.168.123.222/24.

- Ping the MID-360 at 192.168.123.120.

```
ip addr
sudo ip addr add 192.168.123.222/24 dev <your_nic>
ping 192.168.123.120
```

Expected result: the sensor responds to ping and the link stays up.

Step 2. Install Base Build Tools

Goal

Install the tools needed to build Livox SDKs and inspect traffic.

```
sudo apt update
sudo apt install git build-essential cmake tcpdump
```

Step 3. Build and Install Livox-SDK2

Goal

Install the recommended vendor SDK used by the repos main Livox wrapper.

```
cd ~
git clone https://github.com/Livox-SDK/Livox-SDK2.git
cd Livox-SDK2
mkdir build
cd build
cmake .. -DCMAKE_BUILD_TYPE=Release
make -j$(nproc)
sudo make install
sudo ldconfig
```

Expected result: the system can locate liblivox_lidar_sdk.so or
liblivox_lidar_sdk_shared.so.

Step 4. Create a Python Environment

Goal

Install Python dependencies used by the local Livox scripts.

```
cd /home/melodse/ef_ws/g1
python3 -m venv .venv
source .venv/bin/activate
```

pip install -r REQUIREMENTS.txt

The local documentation and scripts rely on numpy, open3d, and for SLAM also kiss-icp.

Step 5. Prepare the MID-360 JSON Config

Goal

Make sure the LiDAR is configured to stream packets to your host IP.

The repo already includes a sample file:

/home/melodse/ef_ws/g1/scripts/navigation/slam/mid360_config.json

Current sample contents:


```
{
"MID360": {
"lidar_net_info": {
"cmd_data_port": 56100,
"push_msg_port": 56200,
"point_data_port": 56300,
"imu_data_port": 56400,
"log_data_port": 56500
},
"host_net_info": [
{
"host_ip": "192.168.123.222",
"multicast_ip": "224.1.1.5",
"cmd_data_port": 56101,
"push_msg_port": 56201,
"point_data_port": 56301,
"imu_data_port": 56401,
"log_data_port": 56501
}
]
}
}
```


If your PC uses a different address, update host_ip and any related host-side IP fields in the configuration you actually launch with.

Step 6. Verify Firewall and UDP Ports

Goal

Prevent the host from dropping Livox packet traffic.

- Allow UDP ports 56100 through 56500 used by the MID-360 config.

- If SDK packet flow stalls, the local docs also suggest checking broader UDP ranges 55000-56000 and 7500-7510.

Step 7. Test Raw Packet Arrival

Goal

Confirm that the LiDAR is actually sending UDP data to the host.

sudo tcpdump -i <your_nic> udp port 56300 -n -c 5

Expected result: you see UDP packets arriving from the MID-360 path.

Step 8. Run the Python Live Viewer

Goal

Bring up a direct point-cloud preview before attempting SLAM.

- Activate the Python environment.

- Change into the Livox SLAM area if the config file is expected there.

- Run the viewer script.

```
cd /home/melodse/ef_ws/g1
source .venv/bin/activate
cd scripts/navigation/slam
python3 ../../sensors/live_points.py
```

Important runtime behavior from the code:

- The viewer prefers SDK2 and falls back to SDK1 if SDK2 is unavailable.

- The default mount mode is LIVOX_MOUNT=upside_down.

- The viewer keeps a short ring buffer to reduce flicker.

Expected result: an Open3D window appears and shows a live point cloud.

Step 9. Fix Mount Orientation If the Cloud Looks Inverted

Goal

Correct the point cloud if the LiDAR is not mounted in the repos default orientation.

```
export LIVOX_MOUNT=normal
python3 ../../sensors/live_points.py
```

Use normal if your sensor is right-side up. The default in the scripts is upside_down.

Step 10. Run the Local SLAM Demo

Goal

Feed Livox frames into KISS-ICP and visualize pose plus accumulated map.

```
cd /home/melodse/ef_ws/g1
source .venv/bin/activate
cd scripts/navigation/slam
python3 live_slam.py
```

Outdoor mode:

```
LIVOX_PRESET=outdoor python3 live_slam.py
```

Expected result: a live map window opens and grows as the sensor moves.

Step 11. Apply Fixed Tilt Compensation If Needed

Goal

Correct scans when the LiDAR is mounted with a known fixed tilt.

```
LIDAR_TILT_DEG=15 LIDAR_TILT_AXIS=y python3 live_slam.py
```

Supported axes are x, y, and z. The script applies the inverse correction automatically.

Step 12. Understand the Main SLAM Presets

Goal

Choose the right preset before changing lower-level parameters.

ParameterIndoorOutdoor

frame_time0.35 s0.20 s

frame_packets200120

voxel_size0.4 m1.0 m

max_range30 m120 m

downsample_limit5,000,0003,000,000

min_motion0.03 m0.10 m

conv_criterion5e-51e-4

max_iters800500

Step 13. Optional ROS Noetic Driver Setup

Goal

Bring up the official ROS1 driver if you want a /livox/lidar topic.

```
source /opt/ros/noetic/setup.bash
mkdir -p ~/catkin_ws/src
cd ~/catkin_ws/src
git clone https://github.com/Livox-SDK/livox_ros_driver2.git
cd livox_ros_driver2
./build.sh ROS1
cd ~/catkin_ws
catkin_make -DCMAKE_BUILD_TYPE=Release -DROS_EDITION=ROS1
```

Add these lines to ~/.bashrc if you use the driver repeatedly:

```
source /opt/ros/noetic/setup.bash
source ~/catkin_ws/devel/setup.bash
```

Step 14. Configure the ROS Driver JSON

Goal

Point the ROS driver at the correct LiDAR IP and host IP.

```
mkdir -p ~/livox_cfg
cp ~/catkin_ws/src/livox_ros_driver2/config/MID360_config.json ~/livox_cfg/
nano ~/livox_cfg/MID360_config.json
```

Make sure the following values are correct:

- lidar_configs[0].ip = 192.168.123.120

- host_net_info.*_ip = your host IP

- pcl_data_type = 1

Step 15. Launch the ROS Driver

Goal

Publish the Livox stream into ROS.

```
roslaunch livox_ros_driver2 msg_MID360.launch \
user_config_path:=/home/$USER/livox_cfg/MID360_config.json \
xfer_format:=0 data_src:=0
```

Expected log lines:

```
[ INFO] Connect lidar: 192.168.123.120 ...
[ INFO] Init lidar success
```

Step 16. Verify the ROS Topic

Goal

Confirm that the ROS driver is publishing point clouds.

rostopic hz /livox/lidar

Expected result: roughly 10 Hz according to the local quick-start guide.

Step 17. Troubleshoot Common Failures

Goal

Use the shortest path to isolate the failure.

- No ping response: recheck cable, NIC selection, and subnet assignment.

- No UDP packets in tcpdump: recheck LiDAR config, firewall, and physical link.

- SDK shared library not found: reinstall SDK2 or set LIVOX_SDK2_LIB or LIVOX_SDK2_DIR.

- Viewer opens but looks empty: zoom out or reset the camera.

- Cloud is upside down: set LIVOX_MOUNT=normal.

- SLAM script exits on KISS-ICP import: install or upgrade kiss-icp.

- ROS bind failed: your host IP in the JSON is wrong or the ports are busy.

- Cannot import rospy: source the ROS setup files before launching the driver.

Step 18. Local Reference Paths

Files You Will Most Likely Revisit

- /home/melodse/ef_ws/g1/docs/lidar_cheatsheet.html

- /home/melodse/ef_ws/g1/docs/lidar_docs.html

- /home/melodse/ef_ws/g1/scripts/navigation/slam/livox2_python.py

- /home/melodse/ef_ws/g1/scripts/navigation/slam/live_slam.py

- /home/melodse/ef_ws/g1/scripts/sensors/live_points.py

- /home/melodse/ef_ws/g1/scripts/navigation/slam/mid360_config.json